

Algoritmos e Estrutura de Dados III

Listas

Aula – 06

Thiago Naves

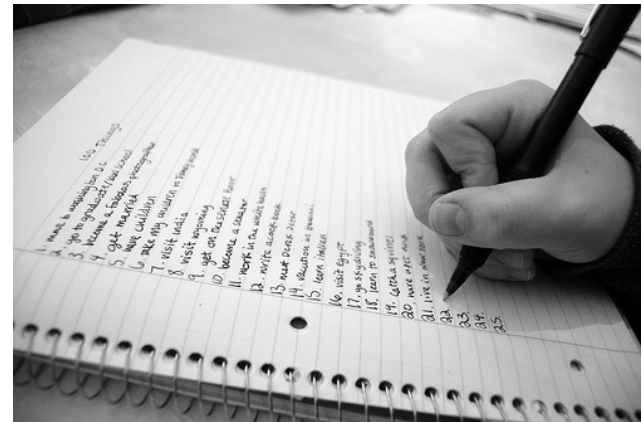
Universidade Tecnológica Federal do Paraná

Bacharelado em Ciência da Computação

Listas

LISTA

*Sem ordem específica de
Inserção ou Remoção*



O que define uma lista

- Uma lista é uma estrutura que armazena elementos de forma alinhada, ou seja, com elementos dispostos um após o outro.
- Para que um elemento seja inserido ou retirado da lista, ele poderá fazer isso por ambas as extremidades (início, fim) e por posições intermediárias (meio).



Estratégia Lista

- ***Sem posições específicas para inserção ou remoção***
- Os elementos da lista podem ser inseridos no início, fim e no meio.
 - Para inserir no final da lista é simples, basta ter espaço na lista para o elemento ser inserido.
 - Contudo, inserção no início e no fim pode ser complexa.
- Caso já exista um elemento no início da lista o mesmo deve ser deslocado para a frente para que o novo elemento possa ser inserido.
- Caso exista um elemento no meio é preciso deslocar todos os elementos da frente para encaixar o novo valor.

Implementações Lista

- Vamos primeiro observar a lista representada de forma estática, com o uso de um vetor.

```
int v[10];
```

```
int n;
```

0	1	2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---	---

- V – representa o vetor
- n – quantidade de elementos que a lista possui

Implementações Lista

Inserir(A, O)

- Operações:
 - Criar uma fila
 - Inserir um elemento na posição x
 - Remover o elemento na posição y
 - Recuperar o elemento na posição z
 - Inserir ou remover elementos no início ou no fim da lista
 - Recuperar o tamanho da lista
 - Destruir uma fila

A

Começo da lista

Implementações Lista

- Operações:
 - Criar uma fila
 - Inserir um elemento na posição x
 - Remover o elemento na posição y
 - Recuperar o elemento na posição z
 - Inserir ou remover elementos no início ou no fim da lista
 - Recuperar o tamanho da lista
 - Destruir uma fila

Inserir(B, O)

B

A

Começo da lista

Implementações Lista

- Operações:
 - Criar uma fila
 - Inserir um elemento na posição x
 - Remover o elemento na posição y
 - Recuperar o elemento na posição z
 - Inserir ou remover elementos no início ou no fim da lista
 - Recuperar o tamanho da lista
 - Destruir uma fila

InserirNoFinal(B)

B

B

A

Começo da lista

Implementações Lista

- Operações:
 - Criar uma fila
 - Inserir um elemento na posição x
 - Remover o elemento na posição y
 - Recuperar o elemento na posição z
 - Inserir ou remover elementos no início ou no fim da lista
 - Recuperar o tamanho da lista
 - Destruir uma fila

Remove(1)

B

B

-

Começo da lista

Listas implementadas com vetor

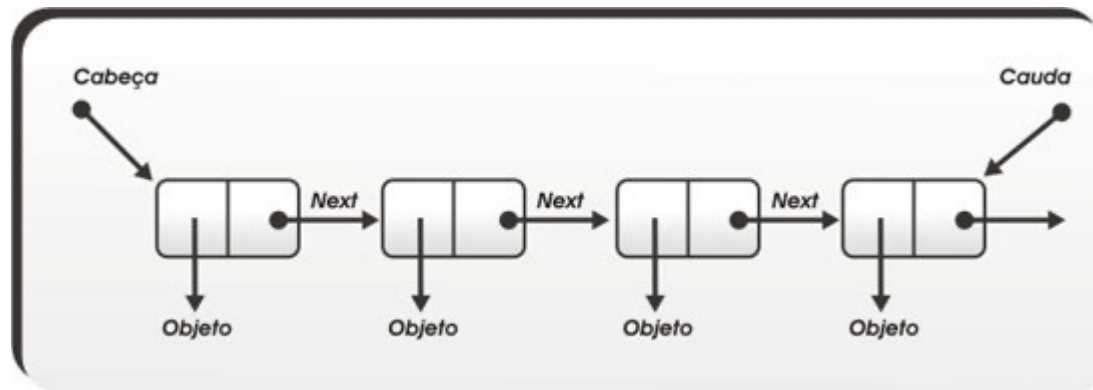
- Criar uma lista $O(1)$
- Inserir/remover no final $O(1)$
- Inserir/remover na posição X $O(N)$
- Recuperar o elemento na posição X $O(1)$
- Recuperar o tamanho da lista $O(1)$
- Destruir a lista $O(1)$
- Resumindo, operações de inserção e remoção são muito lentas e complexas de serem feitas.
- Precisamos de uma nova estratégia de alocação.

Listas alocadas dinamicamente

- Suportam as mesmas operações de criar, inserir/remover no final, inserir/remover no meio, recuperar o valor de um elemento, etc.
- Possuem modo de operação diferente.
- Elementos são armazenados em nós, e cada nó tem um apontador para o próximo nó na lista.
- Iremos definir uma nova forma de representar dados e sua sequencia de armazenamento.
 - Pela semelhança das operações de uma lista e um vetor, podemos entender que iremos definir uma nova forma de representar vetores.

Listas alocadas dinamicamente

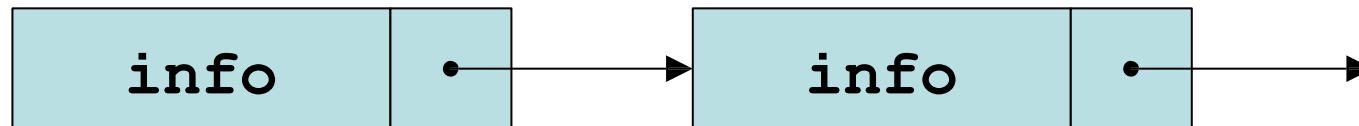
- Lista encadeada



Listas encadeadas

- Essa será uma lista de inteiros, devido ao tipo da variável *info*

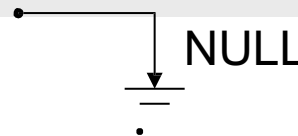
```
struct elemento {  
    int info;  
    struct elemento* prox;  
};  
typedef struct elemento Elemento;  
typedef struct elemento* Lista;
```



Listas encadeadas

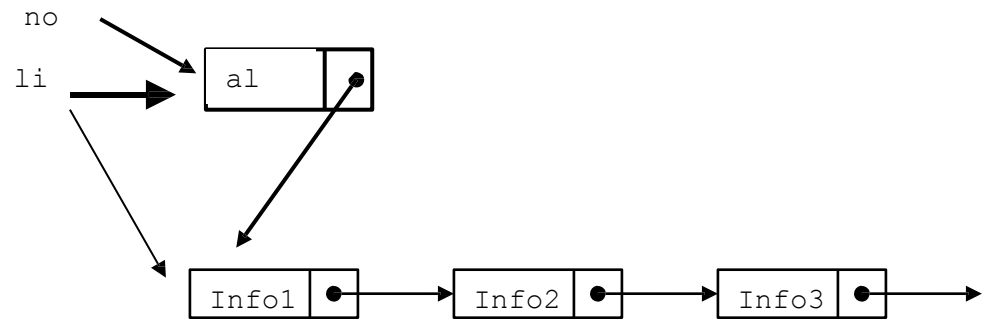
- Essa será uma lista de inteiros, devido ao tipo da variável *info*

```
/* função de criação: retorna uma lista vazia */  
Lista* cria_lista() {  
    Lista* li = (Lista*) malloc(sizeof(Lista));  
    if(li != NULL)  
        *li = NULL;  
    return li; }
```



Cria uma lista vazia, representada pelo ponteiro NULL

Listas encadeadas



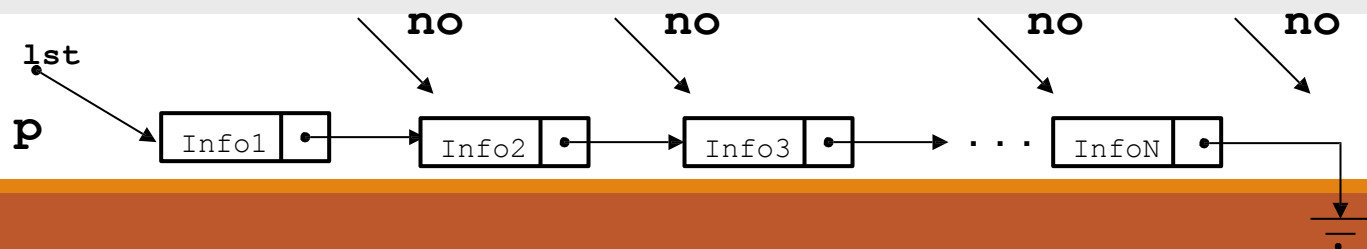
```
/* inserção no início: retorna a lista atualizada */  
int insere_lista_inicio(Lista* li, int num){  
    if(li == NULL)  
        return 0;  
    Elemento* no;  
    no = (Elemento*) malloc(sizeof(Elemento));  
    if(no == NULL)  
        return 0;  
    no->info= num;  
    no->prox = (*li);  
    *li = no;  
    return 1;  
}
```

Listas encadeadas

- Imprime a lista. Duas formas de fazer

```
void lst_imprime (Lista* li)
{
    Elemento* no = *li;
    while(no != NULL){
        printf(("info = %d\n", no->info);
        no = no->prox;
    }
}
```

```
void lst_imprime (Lista* li)
{
    Elemento* no;
    for (no = li; no != NULL; no = no->prox)
        printf("info = %d\n", no->info);
}
```



Listas encadeadas

- Lista vazia.

```
int lista_vazia(Lista* li){  
    if(li == NULL)  
        return 1;  
    if(*li == NULL)  
        return 1;  
    return 0;  
}
```

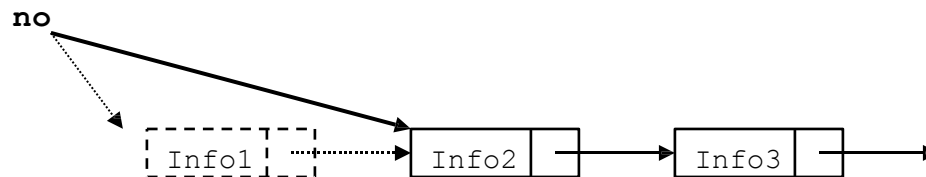
Listas encadeadas

- Consulta um valor.

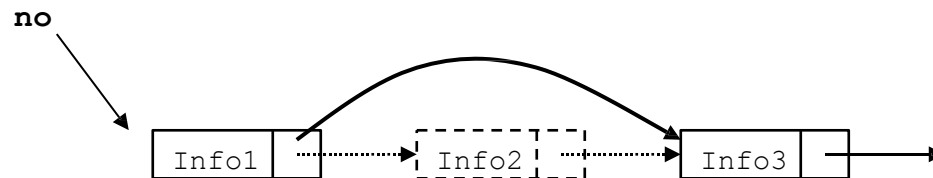
```
int consulta_lista(Lista* li, int num, int *resul){
    if(li == NULL)
        return 0;
    Elemento *no = *li;
    while(no != NULL && no->info != num){
        no = no->prox;
    }
    if(no == NULL)
        return 0;
    else{
        *resul = no->info;
        return 1;
    }
}
```

Listas encadeadas

- Remover um elemento.
 - Recebe como entrada a lista e o valor do elemento a retirar
 - Atualiza o valor da lista, se o elemento removido for o primeiro



- Caso contrário, apenas remove o elemento da lista



Listas encadeadas

- Remover um elemento.

```
int remove_lista(Lista* li, int num){
    if(li == NULL)
        return 0;
    if((*li) == NULL)//lista vazia
        return 0;
    Elemento *ant, *no = *li;
    while(no != NULL && no->info != num){
        ant = no; //ponteiro para o nó anterior ao que sera removido
        no = no->prox;
    }
    if(no == NULL)//não encontrado
        return 0;

    if(no == *li)//remover o primeiro?
        *li = no->prox;
    else
        ant->prox = no->prox;
    free(no);
    return 1;
}
```

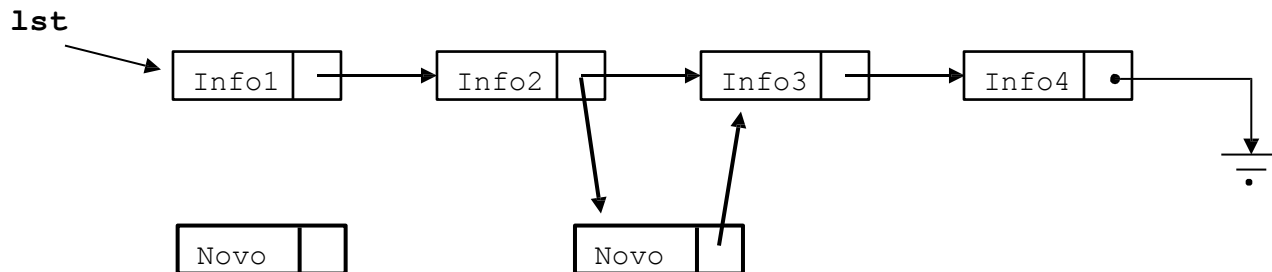
Listas encadeadas

- Liberar a lista destruindo os nós

```
void libera_lista(Lista* li){
    if(li != NULL){
        Elemento * no;
        while((*li) != NULL){
            no = *li;
            *li = (*li)->prox;
            free(no);
        }
        free(li);
    }
}
```

Listas encadeadas

- Manutenção para uma lista ordenada
- Inserir os elementos na ordem correta
- Função de inserção percorre os elementos da lista até encontrar a posição correta para a inserção do novo



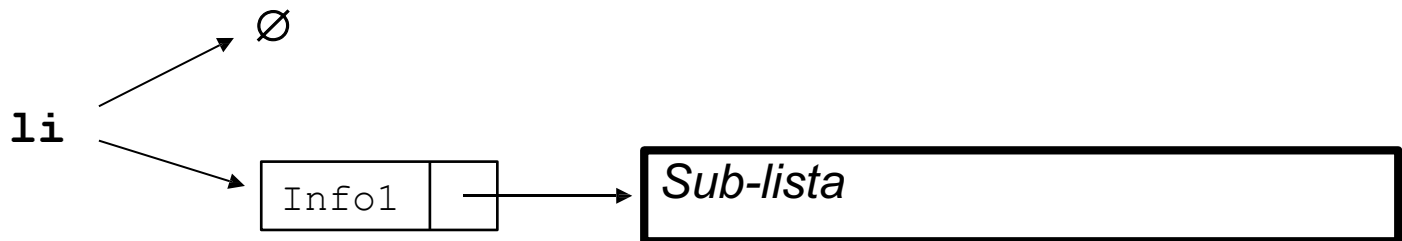
Listas encadeadas

- Insere o elemento ordenado.

```
int insere_lista_ordenada(Lista* li, int num){
    if(li == NULL)
        return 0;
    Elem *no = (Elem*) malloc(sizeof(Elem));
    no->info = num;
    Elem *ant, *atual = *li;
    while(atual != NULL && atual->info < num){
        ant = atual;
        atual = atual->prox;
    }
    if(atual == *li){//insere início
        no->prox = (*li);
        *li = no;
    }else{
        no->prox = atual;
        ant->prox = no;
    }
    return 1;
}
```

Listas encadeadas recursivas

- Imprime uma lista de forma recursiva
- Se a lista for vazia, não imprima nada, caso contrário,
 - Imprima a informação associada ao primeiro nó, dada por `li->info`
 - Imprima a sub-lista, dada por `lst->prox`, chamando recursivamente a função



Listas encadeadas recursivas

```
/* Função imprime recursiva */
void lst_imprime_rec (Lista* li)
{
    if ( ! lst_vazia(li)) {
        /* imprime primeiro elemento */
        printf("info: %d\n",li->info);
        /* imprime sub-lista */
        lst_imprime_rec(li->prox);
    }
}
```

Listas encadeadas recursivas

```
/* Função imprime recursiva invertida */
void lst_imprime_rec (Elemento* li)
{
    if ( !lst_vazia(li) ) {
        /* imprime sub-lista */
        lst_imprime_rec(li->prox);
        /* imprime ultimo elemento */
        printf("info: %d\n",li->info);
    }
}
```

Listas encadeadas com estruturas

- Exemplo – Lista de retângulos

```
struct retangulo {  
    float b;  
    float h;  
};  
typedef struct retangulo Retangulo;
```

```
struct elemento {  
    Retangulo* info;   
    struct elemento *prox;  
};  
typedef struct elemento Elemento;
```

campo da informação representado
por um ponteiro para uma estrutura,
em lugar da estrutura em si

Listas encadeadas com estruturas

- Exemplo – Função auxiliar para alocar um nó

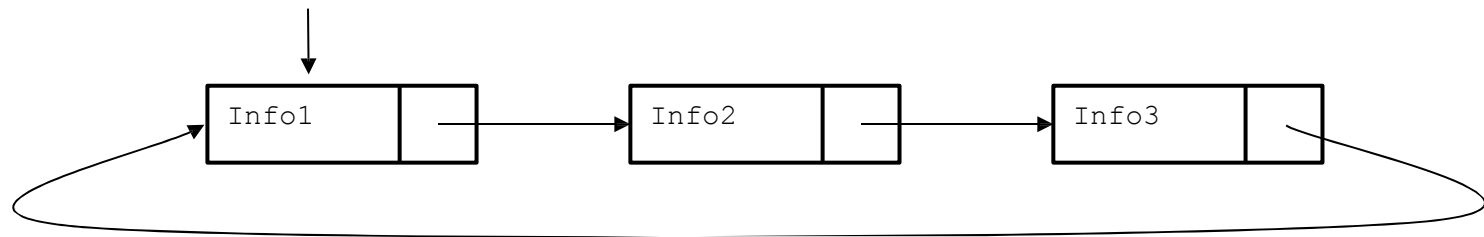
```
static Elemento* aloca (float b, float h)
{
    Retangulo* r = (Retangulo*) malloc(sizeof(Retangulo));
    Elemento* p = (Elemento*) malloc(sizeof(Elemento));
    r->b = b;
    r->h = h;
    p->info = r;
    p->prox = NULL;
    return p;
}
```

Para alocar um nó, são necessárias duas alocações dinâmicas: uma para criar a estrutura do retângulo e outra para criar a estrutura do nó.

O valor da base associado a um nó **p** seria acessado por: **p->info->b**.

Listas encadeadas circulares

- Lista circular:
 - O último elemento tem como próximo o primeiro elemento da lista, formando um ciclo
 - A lista pode ser representada por um ponteiro para um elemento inicial qualquer da lista



Listas encadeadas circulares

- Exemplo - Função para imprimir uma lista circular
 - visita todos os elementos a partir do ponteiro do elemento inicial até alcançar novamente esse mesmo elemento
 - se a lista é vazia, o ponteiro para um elemento inicial é NULL

```
/* função imprime: imprime valores dos elementos */ void
lccirc_imprime (Elemento* li)
{
    Elemento* p = li;                /* faz p apontar para o nó inicial */
    /* testa se lista não é vazia e então percorre com do-while */
    if (p!=NULL) {
        do {
            printf("%d\n", p->info);    /* imprime informação do nó */
            p = p->prox;                /* avança para o próximo nó */
        } while (p != li);
    }
}
```

Referências Oficiais

- BOAVENTURA NETTO, Paulo Oswaldo. **Grafos: teoria, modelos, algoritmos**. 2011.
- CORMEM, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. **Algoritmos: teoria e prática**. 3. ed. São Paulo, SP: Campus, 2012.
- PEREIRA, Silvio L. **Estrutura de Dados Fundamentais: Conceitos e Aplicações**. 12. ed. São Paulo, SP: Érica, 2010.

Referências Oficiais

- BOAVENTURA NETTO, Paulo O. JURKIEWICS, Samuel. **Grafos: Introdução e prática**. 5. ed. São Paulo, SP: Blucher, 2012.
- FORBELLONE, André L. V.; EBERSPASHER, Henri F. **Lógica de programação: a construção de algoritmos e estruturas de dados**. 3. ed. São Paulo, SP: Prentice Hall, 2005.
- GOLDBARG, Marco; GOLDBARG, Elizabeth. **Grafos: Conceitos, Algoritmos e Aplicações**. 1. ed. São Paulo, SP: Campus, 2012.
- SZWARCFITER, Jayme L.; MARKENSON, Lilian. **Estruturas de dados e seus algoritmos**. 3. ed. Rio de Janeiro, RJ: LTC, 2010.
- TOSCANI, Laira V.; VELOSO, Paulo A. S. **Complexidade de algoritmos :análise, projeto e métodos**. 3. ed. Porto Alegre, RS: Bookman, 2012.